EE 5239 Nonlinear Optimization

Lecture 4: First Order Methods: Examples

Mingyi Hong / Jiaxiang Li

University Of Minnesota

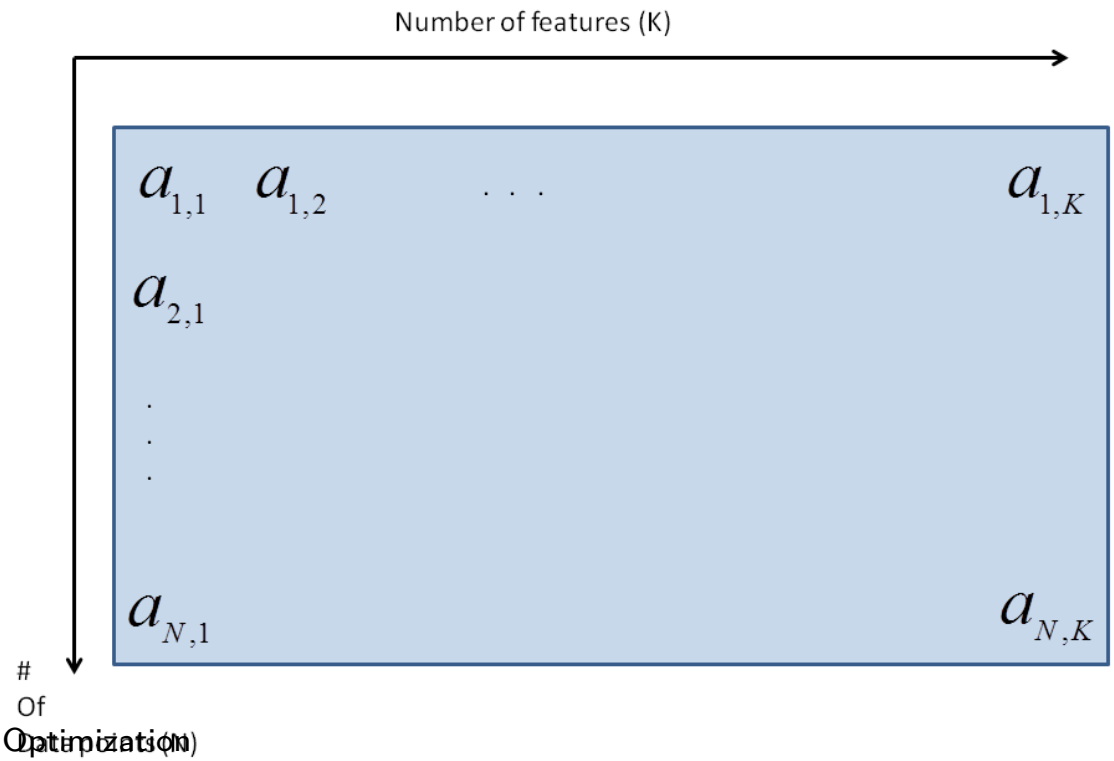
Outline

Last Time

- We talked about a few "first order" methods
- Why they are called first order? Why they are popular?
- Scaled gradient descent – improve condition number
- Incremental gradient method – taking on one data point at each time
- Coordinate descent method – taking on part of the variable at each time
- Notice some interesting things on the names?

Big Data?

- Big Data? problem dimension, or number of data point is huge
- Or both
- Gradient Descent taking on the entire data set together
- Coordinate descent? Incremental?



This Time

- Go over coordinate descent method
- A very useful application of coordinate descent (clustering)
- A very useful application of incremental method (classification)

Outline

Review: Coordinate Descent Method

- Consider $\mathbf{x} = (x_1, \cdots, x_n)$, and

$$\min f(x_1, x_2, \cdots, x_n)$$

- Each coordinate can be either a scalar, or a vector
- For the second case: Block Coordinate Decent, BCD
- Choose the search directions along the coordinate directions
- Iterate through the coordinates cyclically [can have other selection rules]
- **Version 1** (exact minimization): at iteration $r+1$, pick $i = \text{rem}(r, n) + 1$

$$\mathbf{x}_i^{r+1} = \arg\min_{x_i} f(x_1^r, \cdots, x_{i-1}^r, x_i, x_{i+1}^r, \cdots, x_n^r)$$

$$\mathbf{x}_j^{r+1} = \mathbf{x}_j^r, \ \forall\ j \neq i$$

where $\text{rem}(r, n)$ is the remainder of r/n.

Review: Coordinate Descent Method

- Consider $\mathbf{x} = (x_1, \cdots, x_n)$, and

$$\min f(x_1, x_2, \cdots, x_n)$$

- Choose the search directions alone the coordinate directions
- Iterate through the list of coordinates cyclically
- **Version 2** (gradient step): at iteration $r+1$, pick $i = \text{rem}(r, n) + 1$

$$\mathbf{x}_i^{r+1} = \mathbf{x}_i^r - \alpha_i^r \nabla_i f(x^r)$$
$$\mathbf{x}_j^{r+1} = \mathbf{x}_j^r, \ \forall\ j \neq i$$

  where $\text{rem}(r, n)$ is the remainder of r/n.
- Here $\nabla_i f(x^r)$ means taking the gradient with respect to the ith coordinate
- Often known as **(Block) Coordinate Gradient Descent** (BCGD)

How to pick the stepsize?

- **Version 2** (gradient step): at iteration $r+1$, pick $i = rem(r, n) + 1$

$$\mathbf{x}_i^{r+1} = \mathbf{x}_i^r - \alpha_i^r \nabla_i f(x^r)$$
$$\mathbf{x}_j^{r+1} = \mathbf{x}_j^r, \ \forall \ j \neq i$$

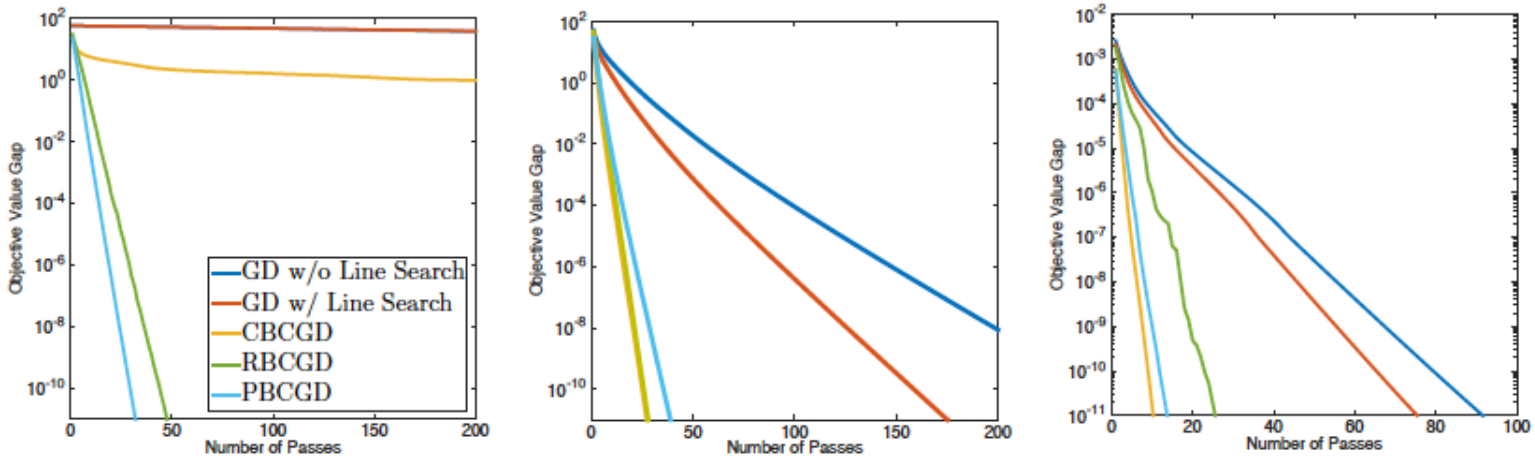
- How to pick the stepsize α_i^r? Same as GD, but....
- Consider we pick a constant stepsize, then each block has stepsize α_i
- Related to the "maximum curvature " of the ith subproblem (refers to as L_i)!
- Usually $L_i << L$, therefore, Large Steps ! [example]

Coordinate Selection Rules

- There are a lot of possible rules for picking coordinates
- Numerically they can exhibit significant difference
- Examples:
  1. **Cyclic**: $1, 2, \cdots, n, 1, 2, \cdots, n$
  2. **Randomized**: each coordinate has probability $1/n$ of being selected (sample with replacement)
  3. **Permuted**: each "length n sequence" has the same probability of being picked (sample without replacement)

Performance

- BCD v.s. gradient descent (GD) for least squares problem



(a) Setting (I) (b) Setting (II) (c) Setting (III)

Figure 1: Comparison among different methods under different settings. "RBCGD" and "PBCGD" denote the randomized BCD-type and permuted BCD-type methods respectively. The vertical axis corresponds to the gap towards the optimal objective value, $\log[\mathcal{F}(x) - \mathcal{F}(x^*)]$; the horizontal axis corresponds to the number of passes over p blocks of coordinates. Though all methods attain linear iteration complexity, their empirical behaviors are different from each others. Note that in plot (b) the curves for the CBCGD method and the RBCGD methods overlap.

Outline

K-Means Clustering: Setting

- Identifying "groups" or "clusters" from unlabeled data

K-Means Clustering: Setting

- Identifying "groups" or "clusters" from unlabeled data
- Consider M data points $\{\mathbf{a}_1, \cdots, \mathbf{a}_M\}$, $\mathbf{a}_i \in \mathbb{R}^K$ (feature space, K-dimensional)
- Suppose we want to divide them into P clusters
- Associate a "mean" or cluster center for each cluster μ_p

K-Means Clustering: Setting

- Identifying "groups" or "clusters" from unlabeled data
- Consider M data points $\{\mathbf{a}_1, \cdots, \mathbf{a}_M\}$, $\mathbf{a}_i \in \mathbb{R}^K$ (feature space, K-dimensional)
- Suppose we want to divide them into P clusters
- Associate a "mean" or cluster center for each cluster μ_p
- Define r_{mp} be indicator of $\mathbf{a}_m \in C_p$ (taking the value 0, or 1)
  1. $r_{mp} = 1$ if the data point $\mathbf{a}_m$ belongs to cluster p
  2. $r_{mp} = 0$ if the data point $\mathbf{a}_m$ does not belongs to cluster p

K-Means Clustering: Setting

- Formulate the following problem

$$L(r, \mu) = \sum_{m=1}^{M} \sum_{p=1}^{P} r_{mp} \|\mathbf{a}_m - \mu_p\|^2$$

K-Means Clustering: Setting

- Formulate the following problem

$$L(r, \mu) = \sum_{m=1}^{M} \sum_{p=1}^{P} r_{mp} \|\mathbf{a}_m - \mu_p\|^2$$

- Minimize the total distance between the points and their "cluster centers"
- Decision variables are $r_{mp} \in \{0, 1\}$ and μ_p's
- That is, the decision variables are the "cluster membership" and the "cluster centers"

K-Means Clustering: Idea

- Optimization is performed by using "block" coordinate descent
- First block coordinates $\{r_{mp}\}$, second block coordinate $\{\mu_p\}$
- Initialize with arbitrary $\{\mu_p\}$
- Iterative updates until convergence
- As we have seen before, objective always decreases and converges

K-Means Clustering: Steps

- **Update** r_{mp}: Assign $\mathbf{a}_i$ to nearest cluster means μ_p

$$r_{mp} = 1, \text{ if } p = \arg\min_j \|\mathbf{a}_m - \mu_j\|,$$

$$r_{mp} = 0, \text{ else}$$

K-Means Clustering: Steps

- **Update** r_{mp}: Assign $\mathbf{a}_i$ to nearest cluster means μ_p

$$r_{mp} = 1, \text{ if } p = \arg\min_j \|\mathbf{a}_m - \mu_j\|,$$

$$r_{mp} = 0, \text{ else}$$

- **Update for** μ_p: Optimize objective w.r.t. μ_p get

$$\mu_p = \frac{\sum_m r_{mp}\mathbf{a}_m}{\sum_m r_{mp}} = \frac{\sum_{x_m \in C_p} \mathbf{a}_m}{|C_p|}$$

where $|C_p|$ means the number of elements in the cluster p; Basically performs an average over all the current class p data points

K-Means Clustering: Example

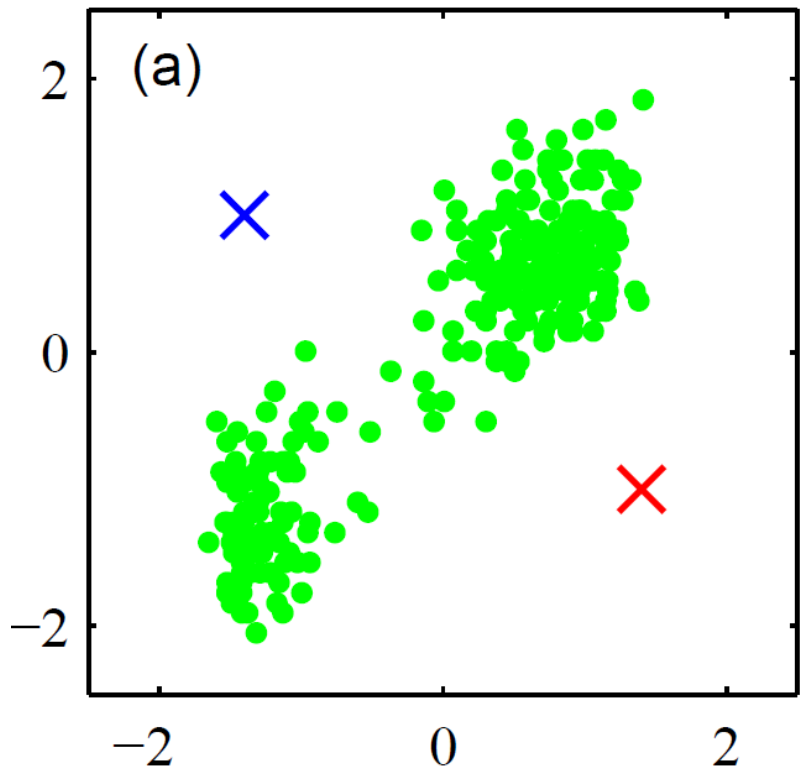


Figure 0.1: Example of K-means [A. Banerjee 13].

K-Means Clustering: Example

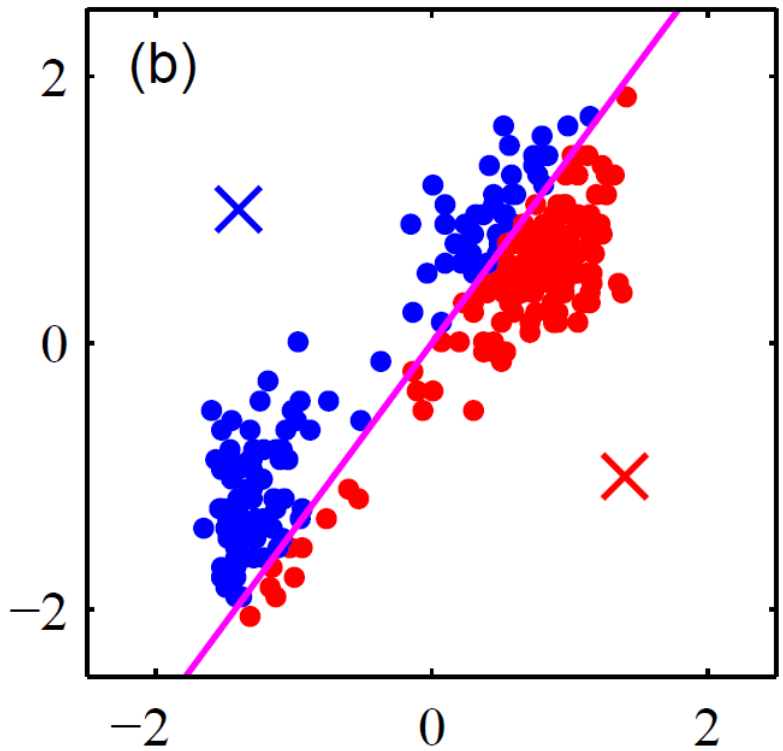


Figure 0.2: Example of K-means [A. Banerjee 13].

K-Means Clustering: Example

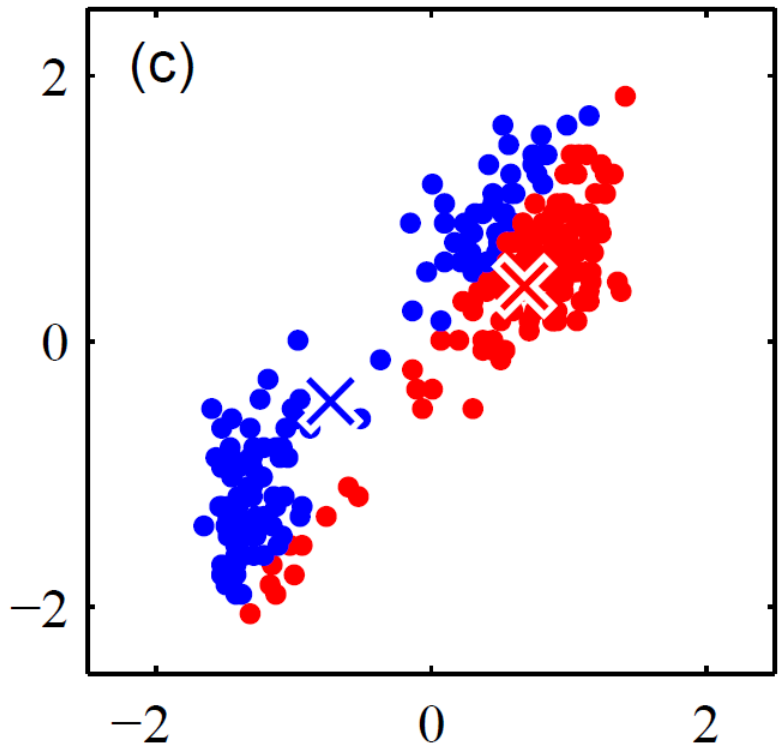


Figure 0.3: Example of K-means [A. Banerjee 13].

K-Means Clustering: Example

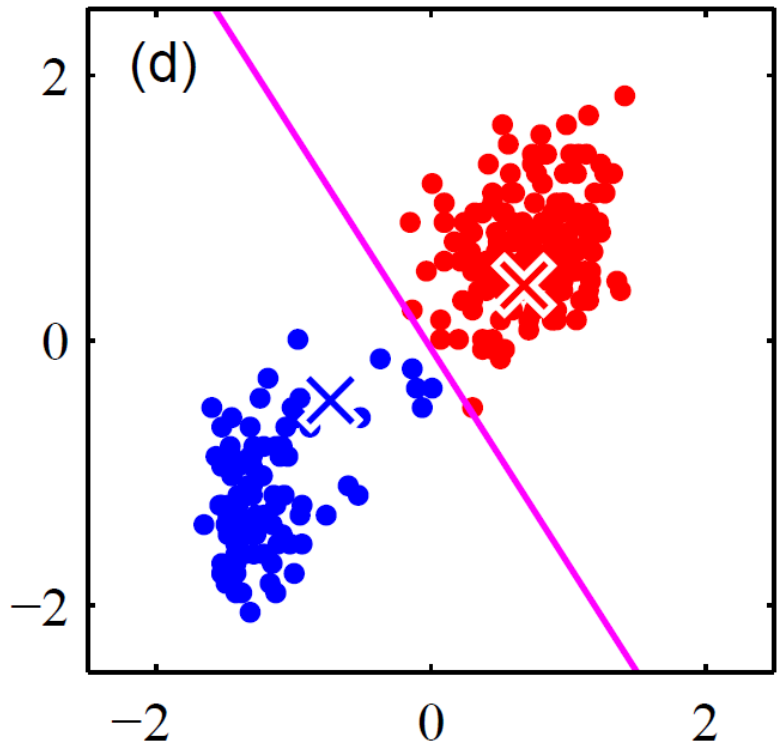


Figure 0.4: Example of K-means [A. Banerjee 13].

K-Means Clustering: Example

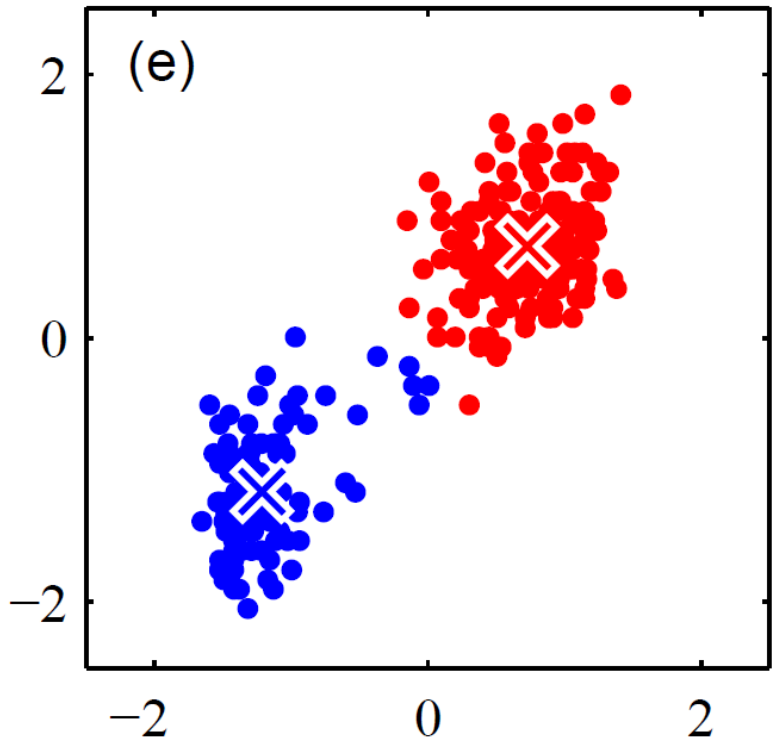


Figure 0.5: Example of K-means [A. Banerjee 13].

K-Means Clustering: Example

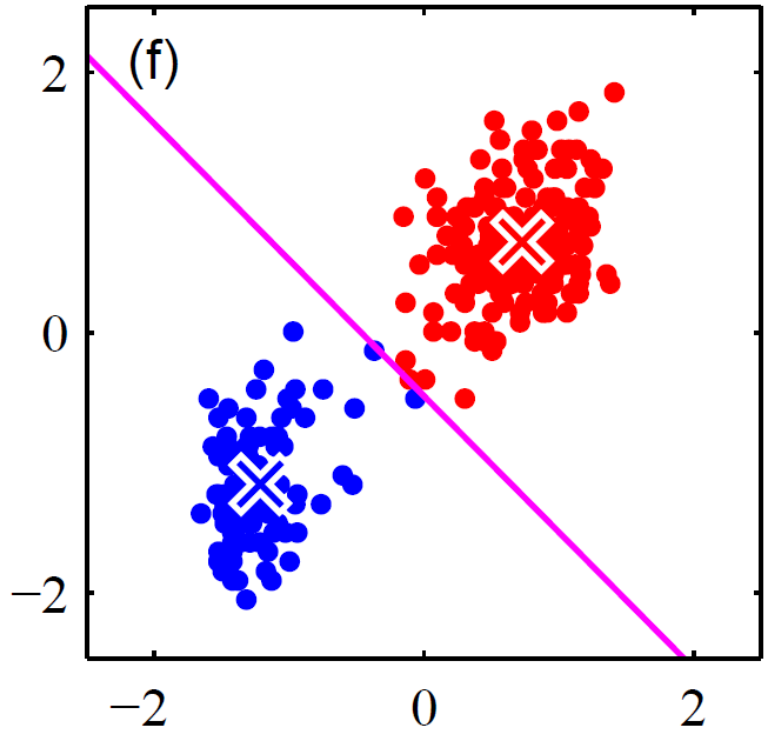


Figure 0.6: Example of K-means [A. Banerjee 13].

K-Means Clustering: Example

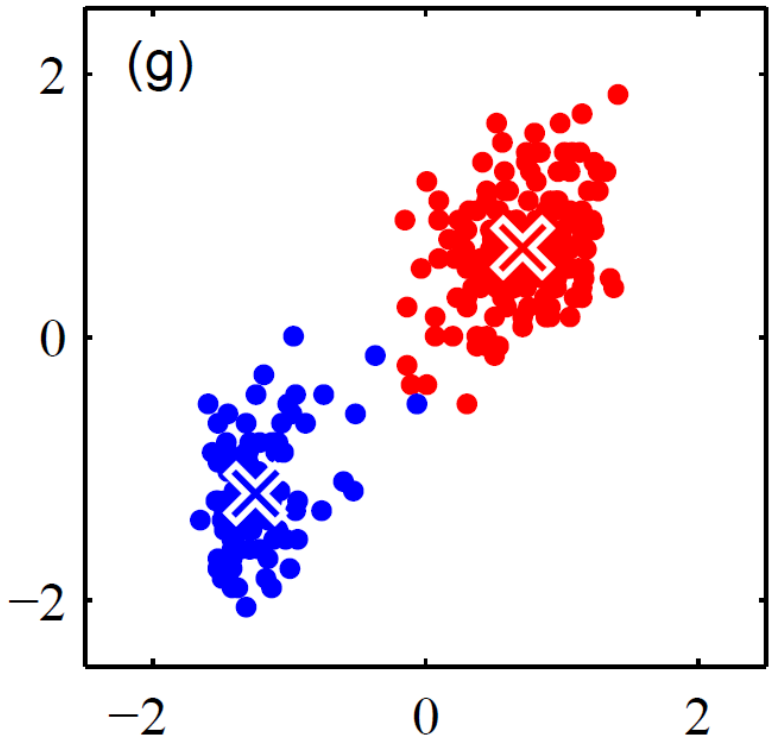


Figure 0.7: Example of K-means [A. Banerjee 13].

K-Means Clustering: Example

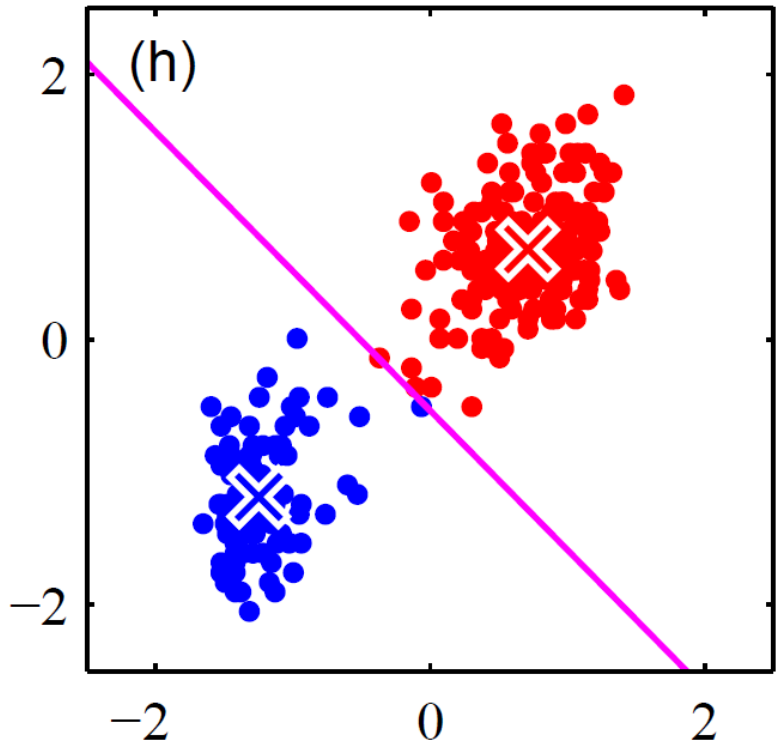


Figure 0.8: Example of K-means [A. Banerjee 13].

K-Means Clustering: Example

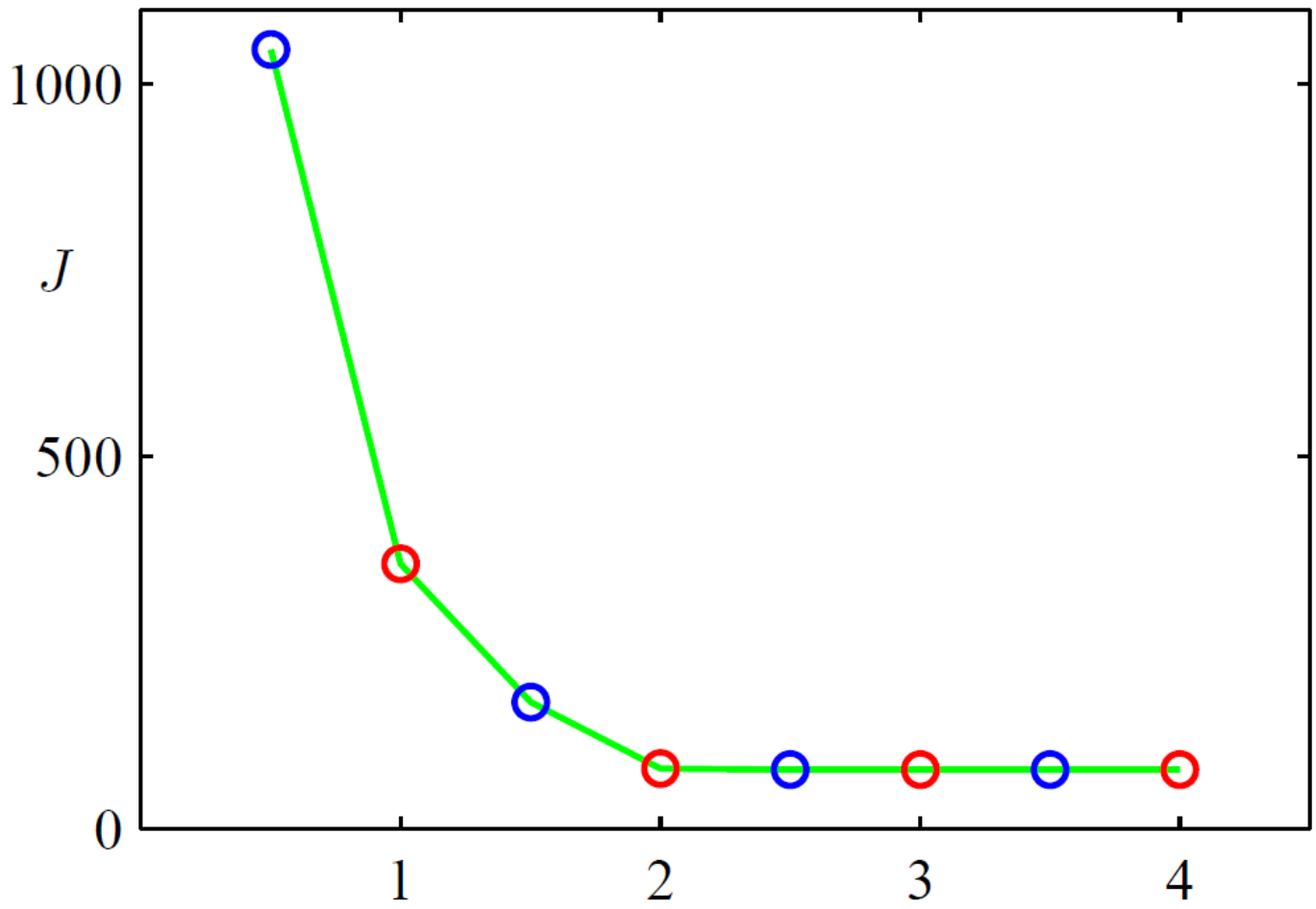


Figure 0.9: Example of K-means: Objective value reduces [A. Banerjee 13].

K-Means Clustering: Example

Figure 0.10: Example of K-means: original image [A. Banerjee 13].

K-Means Clustering: Example

Figure 0.11: Example of K-means: 10 clusters [A. Banerjee 13].

K-Means Clustering: Example

Figure 0.12: Example of K-means: 3 clusters [A. Banerjee 13].

K-Means Clustering: Example

Figure 0.13: Example of K-means: 2 clusters [A. Banerjee 13].

Outline

A simple learning algorithm

- Assuming we have the training data points $\{\mathbf{a}_i, b_i\}_{i=1}^n$, $\mathbf{a}_i \in \mathbb{R}^d$, $b_i \in \{-1,\ 1\}$
- We want to learn the model $\text{sgn}(\mathbf{a}^T\mathbf{x})$ to classify the training data into two different categories
- This is the "classification" problem we discussed at Lecture 0
- Supervised learning approach

A simple learning algorithm "Perceptron"

- Suppose we start at an arbitrary solution $\mathbf{x}$
- Pick a misclassified point: $\text{sgn}((\mathbf{x})^T \mathbf{a}_n) \neq b_n$, or $b_n(\mathbf{x})^T \mathbf{a}_n < 0$
- Update the weight vector by

$$\mathbf{x} \leftarrow \mathbf{x} + b_n \mathbf{a}_n$$

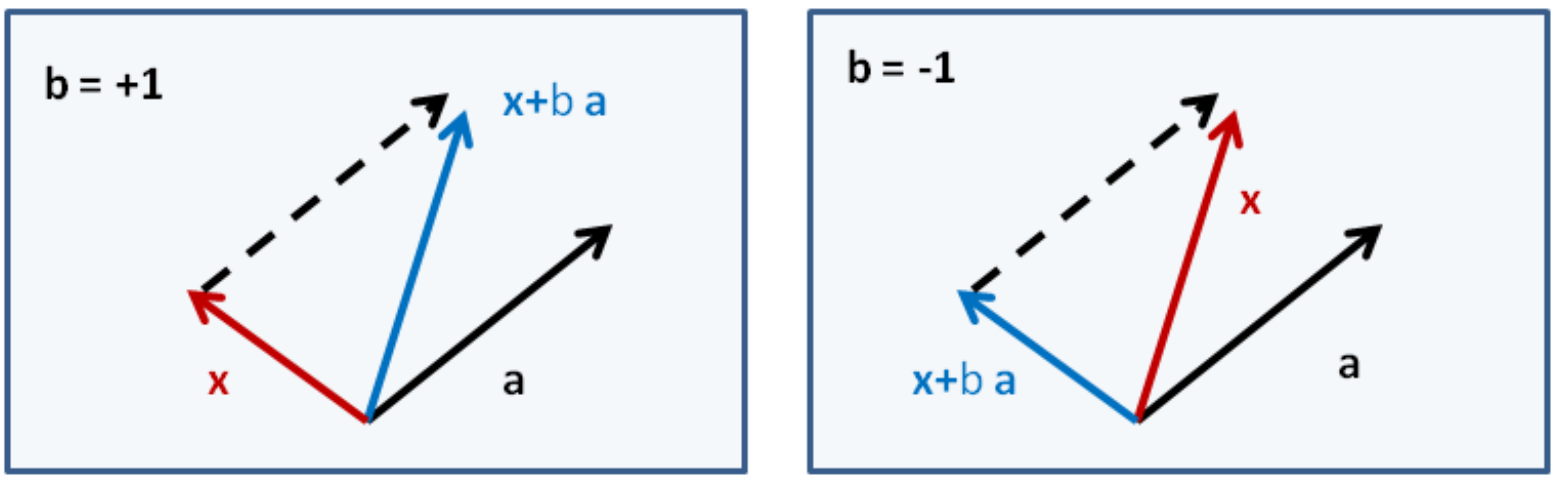


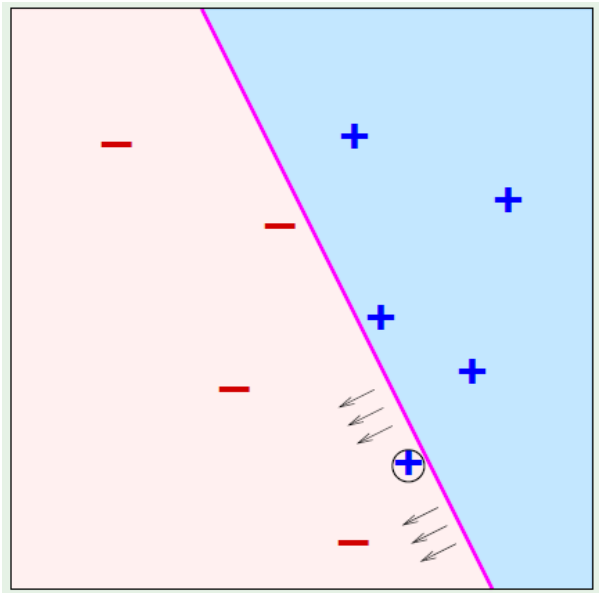
Figure 0.1: The update rules (Y. Abu-Mostafa).

A simple learning algorithm (cont.)

- At iteration $t = 1, 2, \cdots$, continue pick misclassified point from

$$(\mathbf{a}_1, b_1), (\mathbf{a}_2, b_2), \cdots$$

  and keep running the algorithm

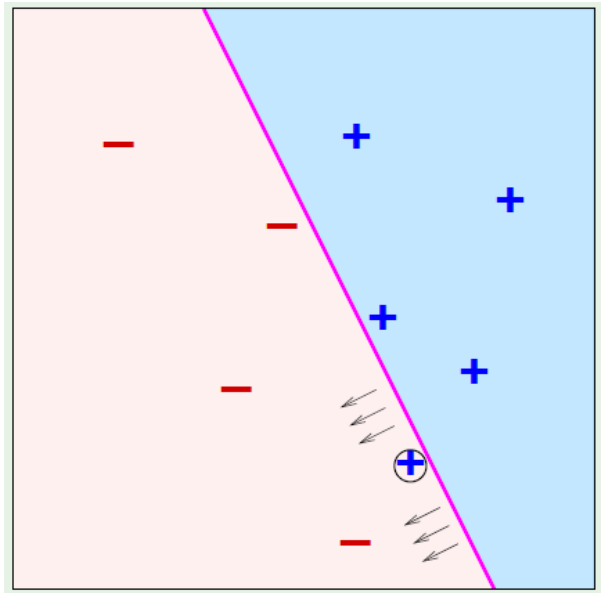


A simple learning algorithm (cont.)

- At iteration $t = 1, 2, \cdots$, continue pick misclassified point from

$$(\mathbf{a}_1, b_1), (\mathbf{a}_2, b_2), \cdots$$

  and keep running the algorithm
- If the training data is linearly separable, then a correct separation plane will be found after finite number of iterations



A simple learning algorithm (cont.)

- A famous algorithm called Peceptron Algorithm
- This algorithm has some nice features
- **Key Feature**: Each time a single data point is processed
- When does it stop? Compare with SVM?
- Looks rather heuristic at this point, but can be viewed as solving certain optimization problem "incrementally" (one data point used at each time)

The Linear Classification Training: Revisited

- Let's consider the objective function (why I am minimizing?)

$$L(\mathbf{x}) = \sum_{i=1}^{m} \max\{-b_i \mathbf{x}^T \mathbf{a}_i, 0\} := \sum_{i=1}^{m} g_i(\mathbf{x})$$

The Linear Classification Training: Revisited

- Let's consider the objective function (why I am minimizing?)

$$L(\mathbf{x}) = \sum_{i=1}^{m} \max\{-b_i \mathbf{x}^T \mathbf{a}_i, 0\} := \sum_{i=1}^{m} g_i(\mathbf{x})$$

- For each miss-classified data point i, $-b_i \mathbf{x}^T \mathbf{a}_i > 0$; the gradient

$$\nabla g_i(\mathbf{x}) = -b_i \mathbf{a}_i$$

The Linear Classification Training: Revisited

- Let's consider the objective function (why I am minimizing?)

$$L(\mathbf{x}) = \sum_{i=1}^{m} \max\{-b_i \mathbf{x}^T \mathbf{a}_i, 0\} := \sum_{i=1}^{m} g_i(\mathbf{x})$$

- For each miss-classified data point i, $-b_i \mathbf{x}^T \mathbf{a}_i > 0$; the gradient

$$\nabla g_i(\mathbf{x}) = -b_i \mathbf{a}_i$$

- For each correct data point i, $-b_i \mathbf{x}^T \mathbf{a}_i < 0$; gradient is zero!

The Linear Classification Training: Revisited

- Let's consider the objective function (why I am minimizing?)

$$L(\mathbf{x}) = \sum_{i=1}^{m} \max\{-b_i \mathbf{x}^T \mathbf{a}_i, 0\} := \sum_{i=1}^{m} g_i(\mathbf{x})$$

- For each miss-classified data point i, $-b_i \mathbf{x}^T \mathbf{a}_i > 0$; the gradient

$$\nabla g_i(\mathbf{x}) = -b_i \mathbf{a}_i$$

- For each correct data point i, $-b_i \mathbf{x}^T \mathbf{a}_i < 0$; gradient is zero!
- Directly applying the incremental method?

$$\mathbf{x}^{r+1} = \mathbf{x}^r + \alpha^r b_i \mathbf{a}_i$$

The Linear Classification Training: Revisited

- Let's consider the objective function (why I am minimizing?)

$$L(\mathbf{x}) = \sum_{i=1}^{m} \max\{-b_i \mathbf{x}^T \mathbf{a}_i, 0\} := \sum_{i=1}^{m} g_i(\mathbf{x})$$

- For each miss-classified data point i, $-b_i \mathbf{x}^T \mathbf{a}_i > 0$; the gradient

$$\nabla g_i(\mathbf{x}) = -b_i \mathbf{a}_i$$

- For each correct data point i, $-b_i \mathbf{x}^T \mathbf{a}_i < 0$; gradient is zero!
- Directly applying the incremental method?

$$\mathbf{x}^{r+1} = \mathbf{x}^r + \alpha^r b_i \mathbf{a}_i$$

- **Good News**: $\alpha^r = 1$!

The Convergence

- Define $\mathbf{u}$, with $\|\mathbf{u}\| = 1$, be an optimal solution, i.e.,

$$b_i \mathbf{u}^T \mathbf{a}_i \geq \gamma > 0, \ \forall \ i$$

  where γ is the largest constant that satisfies the above inequality

The Convergence

- Define $\mathbf{u}$, with $\|\mathbf{u}\| = 1$, be an optimal solution, i.e.,

$$b_i \mathbf{u}^T \mathbf{a}_i \geq \gamma > 0, \ \forall \ i$$

  where γ is the largest constant that satisfies the above inequality

- Let $R := \max_i \|\mathbf{a}_i\|$

The Convergence

- Define $\mathbf{u}$, with $\|\mathbf{u}\| = 1$, be an optimal solution, i.e.,

$$b_i \mathbf{u}^T \mathbf{a}_i \geq \gamma > 0, \ \forall \ i$$

  where γ is the largest constant that satisfies the above inequality

- Let $R := \max_i \|\mathbf{a}_i\|$

- **Claim**: If the data points are separable, and we start with $\mathbf{x}_0 = 0$, then training stops after at most $\left(\frac{R}{\gamma}\right)^2$ iterations

- Exact Convergence in Finite Steps!

Proof steps

- Let $\mathbf{x}_1 = 0$
- Let $\mathbf{x}_{k+1}$ denote the vector making the kth mistake

$$\mathbf{x}_{k+1} = \mathbf{x}_k + b_i \mathbf{a}_i$$

- First we see that each update reduces the error contributed by that point (note, for each miss-classified i, $-b_i \mathbf{x}^T \mathbf{a}_i > 0$)

$$-\mathbf{x}_{k+1}^T \mathbf{a}_i b_i = -\mathbf{x}_k^T \mathbf{a}_i b_i - (\mathbf{a}_i b_i)^T \mathbf{a}_i b_i < -\mathbf{x}_k^T \mathbf{a}_i b_i$$

Proof steps

- Let $\mathbf{x}_1 = 0$
- Let $\mathbf{x}_{k+1}$ denote the vector making the kth mistake

$$\mathbf{x}_{k+1} = \mathbf{x}_k + b_i \mathbf{a}_i$$

- First we see that each update reduces the error contributed by that point (note, for each miss-classified i, $-b_i \mathbf{x}^T \mathbf{a}_i > 0$)

$$-\mathbf{x}_{k+1}^T \mathbf{a}_i b_i = -\mathbf{x}_k^T \mathbf{a}_i b_i - (\mathbf{a}_i b_i)^T \mathbf{a}_i b_i < -\mathbf{x}_k^T \mathbf{a}_i b_i$$

- Then we see that $\mathbf{x}_k^T$ and $\mathbf{u}$ (the optimal solution) are getting "closer" (by induction)

$$\begin{aligned} \mathbf{x}_{k+1}^T \mathbf{u} &= \mathbf{x}_k^T \mathbf{u} + b_i \mathbf{u}^T \mathbf{a}_i \\ &\geq \mathbf{x}_k^T \mathbf{u} + \gamma \\ &\geq k\gamma \end{aligned}$$

  The last step utilizes the initial solution $\mathbf{x}_1 = 0$

Proof steps

- But the norm of $\mathbf{x}_{k+1}$ is upper-bounded (why the inequality here?)

$$\begin{aligned}\|\mathbf{x}_{k+1}\|^2 &= \|\mathbf{x}_k\|^2 + 2b_i\mathbf{x}_k^T\mathbf{a}_i + \|\mathbf{a}_i\|^2 \\ &\leq \|\mathbf{x}_k\|^2 + R^2 \\ &\leq kR^2\end{aligned}$$

- Since we assumed $\|\mathbf{u}\| = 1$, we have

$$k\gamma \leq \mathbf{x}_{k+1}^T\mathbf{u} \overset{(i)}{\leq} \|\mathbf{x}_{k+1}\| \leq \sqrt{k}R \tag{0.1}$$

  where (i) is because of Cauchy-Swarchtz inequality
- Then we arrive at

$$\sqrt{k} \leq \frac{R}{\gamma}, \text{ or } k \leq \frac{R^2}{\gamma^2}.$$

What if the data set is NOT separable

- What if the data set is NOT separable?
- Slightly change the previous algorithm (the 'pocket' algorithm)
- Step 0: Set $\hat{\mathbf{x}} = \mathbf{x}^0$
- Step 1: Run the previous algorithm for one iteration obtain $\mathbf{x}^{r+1}$
- Step 2: Evaluate $L(\mathbf{x}^{r+1})$
- Step 3: If $\mathbf{x}^{r+1}$ is better than $\hat{\mathbf{x}}$ in terms of the loss, set $\hat{\mathbf{x}}$ to $\mathbf{x}^{r+1}$

Main Idea, force the loss function to be decreasing!

Conclusion

- Introduced an example of coordinate descent (K-means)
- Introduced an example of incremental method (Perceptron)
- Next time HW will use real date to implement different type of methods